

Module 4: Advanced Types in TypeScript

1. Introduction to Advanced Types

Advanced types allow greater flexibility and safety when working with TypeScript.

2. Union Types

Union types allow a variable to have multiple types.

```
typescript
```

```
let id: number | string;
```

```
id = 101;
```

```
id = "ABC123";
```

3. Intersection Types

Intersection types combine multiple types.

```
typescript
```

```
interface Person {
```

```
  name: string;
```

```
  age: number;
```

```
}
```

```
interface Employee {
```

```
  employeeId: number;
```

```
}
```

```
type Worker = Person & Employee;
```

```
let worker: Worker = { name: "Alice", age: 30, employeeId: 1234 };
```

4. Literal Types

Literal types restrict values to specific constants.

```
typescript
```

```
let direction: "left" | "right" | "up" | "down";
```

```
direction = "left";
```

5. Type Aliases

Type aliases create reusable type definitions.

typescript

```
type ID = number | string;
```

```
let userId: ID = 101;
```

```
let orderId: ID = "XYZ789";
```

6. Mapped Types

Mapped types create new types based on existing ones.

typescript

```
type ReadonlyPerson = { readonly [K in keyof Person]: Person[K] };
```

7. Conditional Types

Conditional types allow dynamic type resolution.

typescript

```
type IsString<T> = T extends string ? "Yes" : "No";
```

```
type Test = IsString<number>; // Output: "No"
```

8. Type Guards

Type guards help narrow down types at runtime.

typescript

```
function isString(value: unknown): value is string {  
  return typeof value === "string";  
}
```

Practice Questions:

1. Define a variable that can store either a number or a string.
2. Create an intersection type for `User` with `Admin` properties.
3. Implement a function that checks if a value is a string using a type guard.
4. Create a mapped type to make all properties of an interface optional.
5. What will be the output of this TypeScript code?

typescript

```
type Check<T> = T extends string ? "Yes" : "No";  
type Result = Check<42>;
```

Practice Answers:

1. Union Type:

```
typescript  
let data: number | string;
```

2. Intersection Type:

```
typescript  
interface User {  
  name: string;  
  age: number;  
}  
interface Admin {  
  role: string;  
}  
type SuperUser = User & Admin;
```

3. Type Guard Function:

```
typescript  
function isString(value: unknown): value is string {  
  return typeof value === "string";  
}
```

4. Mapped Type to make properties optional:

```
typescript  
type Optional<T> = { [K in keyof T]?: T[K] };
```

5. The output of `Check<42>` is `"No"`.